LAB MANUAL

Sample Web Applications

For Asset Tracking and Multimedia Sharing

Practical Lab-09

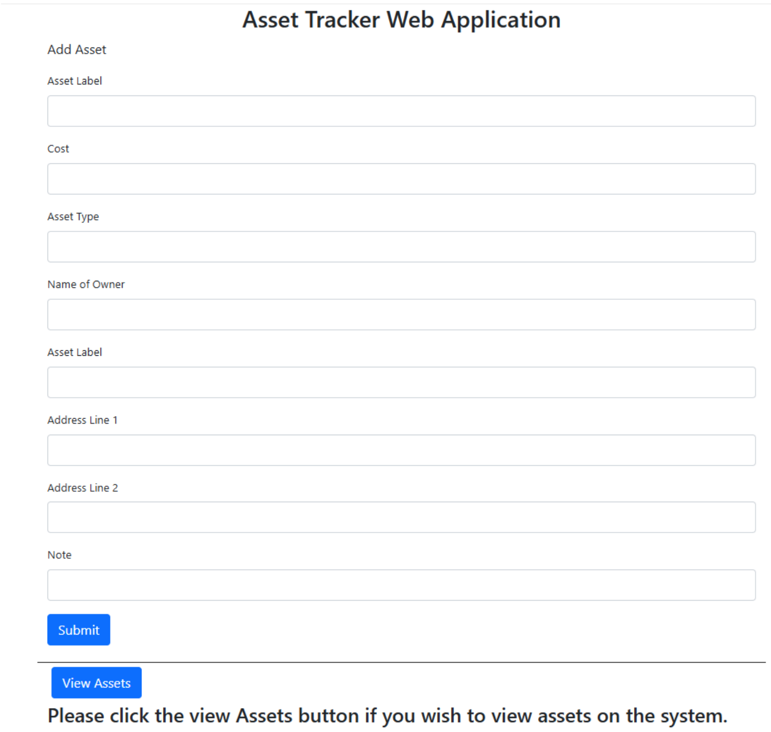
Faculty of Computing, Engineering and The Built Environment

Contents

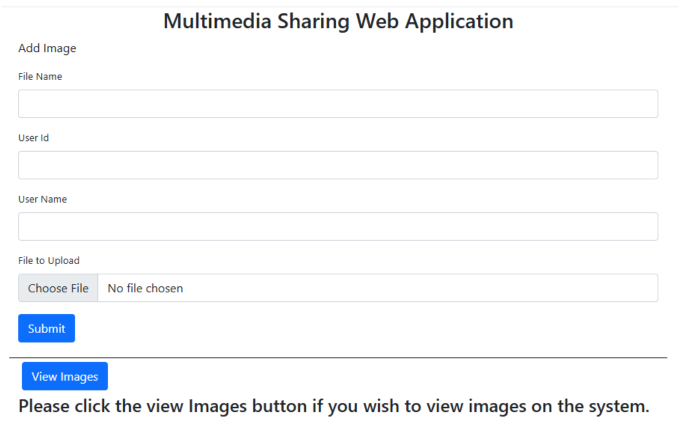
In this lab you will use following two sample web applications. 3
1. Asset Tracker Web Application (Ref to Lab week 8).. 4
WebApp Overview 4
Modifying App.js 6
Providing REST URIs.. 6
2. Multimedia Sharing Web Application (Ref to Lab week 6).. 8
WebApp Overview 8
Modifying App.js 9
Providing REST URIs.. 9
Providing the BASE URI for the storage account 10
Further thoughts 11

In this lab you will use following two sample web applications.

1) Asset Tracker Web Application



2) Multimedia sharing web application



1. Asset Tracker Web Application (Practical Lab week 8)

Prerequisite:

You would need following URIs created in week-8 for using this web application.

1. Creating a POST endpoint (RAA)
2. Creating a POST endpoint (CIA)
3. Creating a DELETE endpoint (DIA)

This web application will use jQuery[1] to provide dynamic functionality and bootstrap.css[2] to provide a reactive grid-based layout.

These tools have been selected due to their maturity and relatively simple functionality. If students are familiar with much more capable frameworks, such as Angular or React, then they may feel free to use this (it is encouraged if you are *au fait*).

In this activity we have provided a very simple, functional, but user-unfriendly web application to help demonstrate how these APIs can be consumed. This web app is designed for education purposes on this topic, not for best practices. Notably in the areas of UX and security.

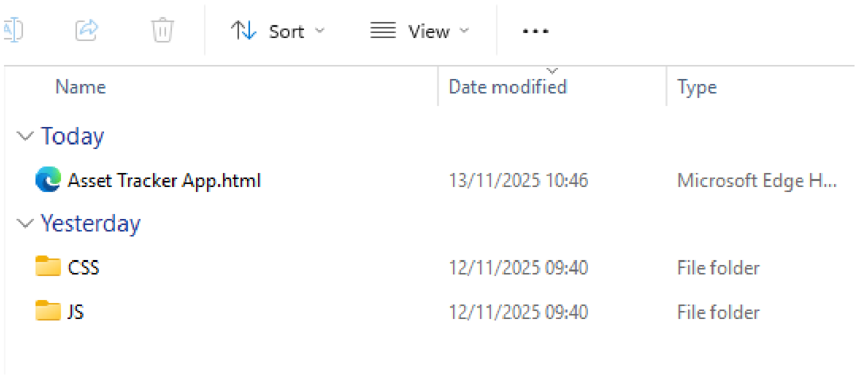
WebApp Overview

This webapp is provided in a zip file "Asset Tracker App" shared via blackboard. It has a simple structure with the following folders and files.

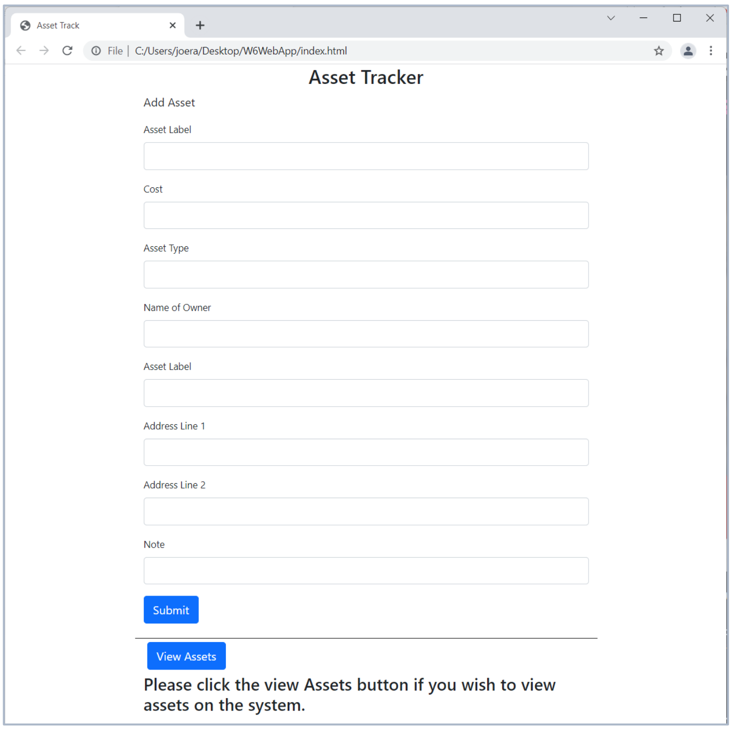
Folder/File	Purpose
CSS	A folder containing CSS files to assist with layout and presentation of the web app. You will typically not need to access the content of this folder.
JS	A folder containing the JavaScript files which power the programmatic aspects of this application, including jQuery libraries. **Notably this has the *app.js* file, which needs to be modified for this practical.**
Asset Tracker App.html	A simple HTML file providing structure to the elements of the web app.

[1] https://jquery.com/ - jQuery
[2] https://getbootstrap.com/ - Bootstrap

Extract the compressed archived to your local file system, a memorable location such as desktop etc.



Open ‘Asset Tracker App.html’ with a web browser.



An Asset Tracker Web application is presented. It has 4 key elements, which are:

1. A form to add new assets to the inventory.
2. A ‘Submit’ button with the ID “subNewForm” to take the contents of the form and send it to the CIA API. This has an event handler specified in the app.js file.
3. A ‘View Assets’ button with the ID “retAssets” to view the assets in the current database by accessing the RAA API and generating HTML on the fly. This has an event handler specified in the app.js file.

4. An Assets display area with the ID “AssetList” this is used to display the HTML generated by the view assets routine.

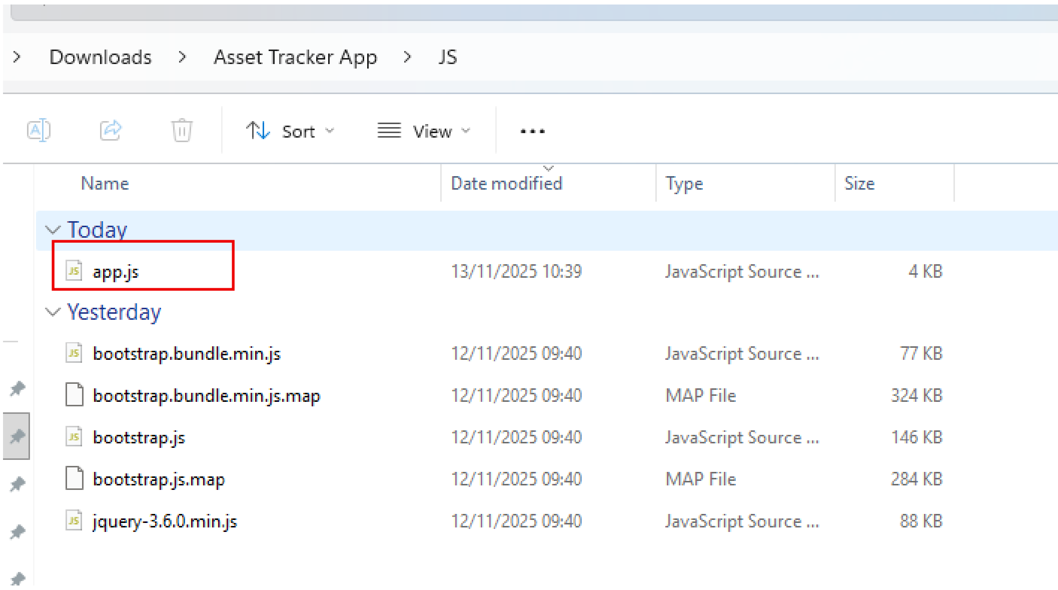
By default, this web app will not work. It needs additional code and variables specified in the app.js file, which has placeholders for the relevant items. Open this file in the code editor of your choice.

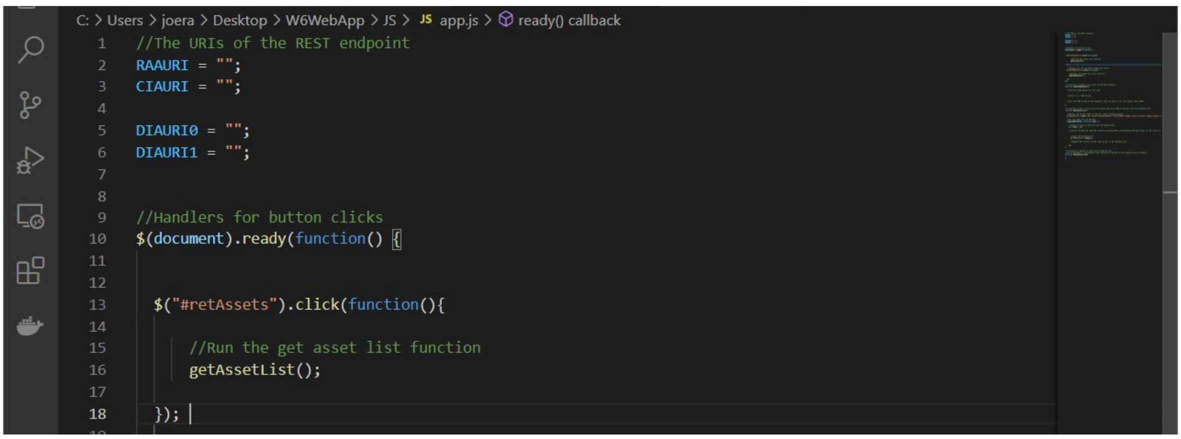
Modifying App.js

Providing REST URIs

Open file ‘app.js’ file in JS folder. You can open this file in notepad++ or VSC, or any file editor of your choice.

It is necessary to add some details about your REST endpoint. These are the URIs for RAA, CIA and DIA. **The necessary lines to be changed are 2,3,5 and 6.** You should have a note of these from the week-8 lab.



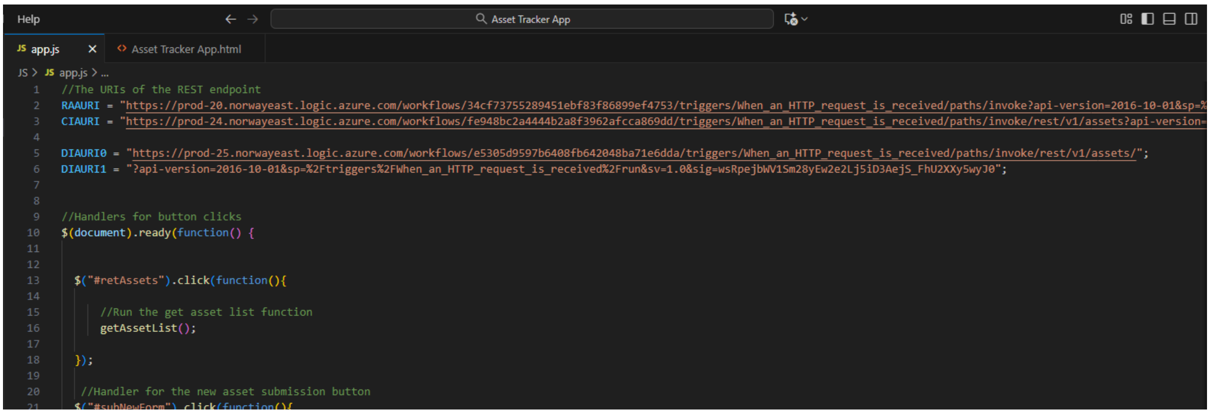


Line 2: RAAURI = This is URI of your RAA logic app.

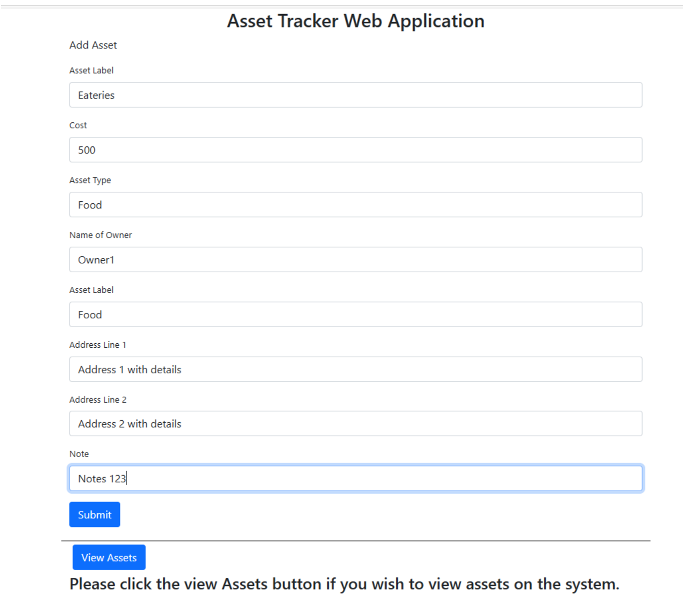
Line 3: CIAURI = This is URI of your CIA logic app.

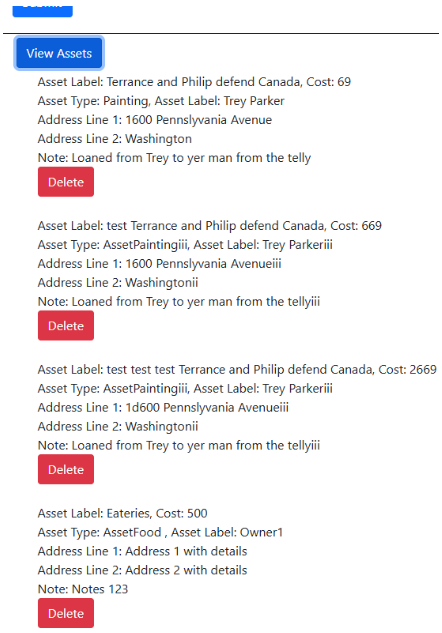
Line 5&6: DIAURI = This is URI of your DIA logic app, divided into two parts. Part1 ending at ‘/asset/’ and other part starting from ‘?api-version…’. This is done to allow Asset ID to be entered in between.

My view for the lines above looks like this:



Now save the app.js file. Open the html file and run the app.





2. Multimedia Sharing Web Application (Ref to Lab week 6)

Prerequisite:

You would need following URIs created in **week-6**.

1. Creating a POST endpoint for Blob and Cosmos (IUPS)
2. Creating a POST endpoint to collect all data (RAI)

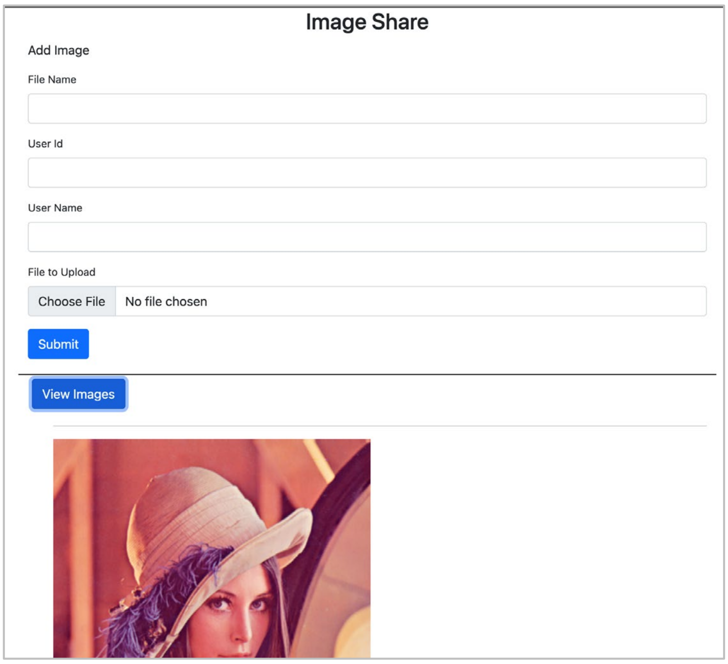
WebApp Overview

This webapp is provided in a zip file "Multimedia Sharing App" shared via blackboard. It has a simple structure with the following folders and files.

Folder/File	Purpose
CSS	A folder containing CSS files to assist with layout and presentation of the web app. You will typically not need to access the content of this folder.
JS	A folder containing the JavaScript files which power the programmatic aspects of this application, including jQuery libraries. **Notably this has the *app.js* file, which needs to be modified for this practical.**
Multimedia App.html	A simple HTML file providing structure to the elements of the web app.

Extract the compressed archived to your local file system, a memorable location such as the desktop is ideal.

Navigate to this folder and locate the Multimedia App.html file, open this with a web browser.



An Image Tracker Web application is presented. It has 4 key elements, which are:

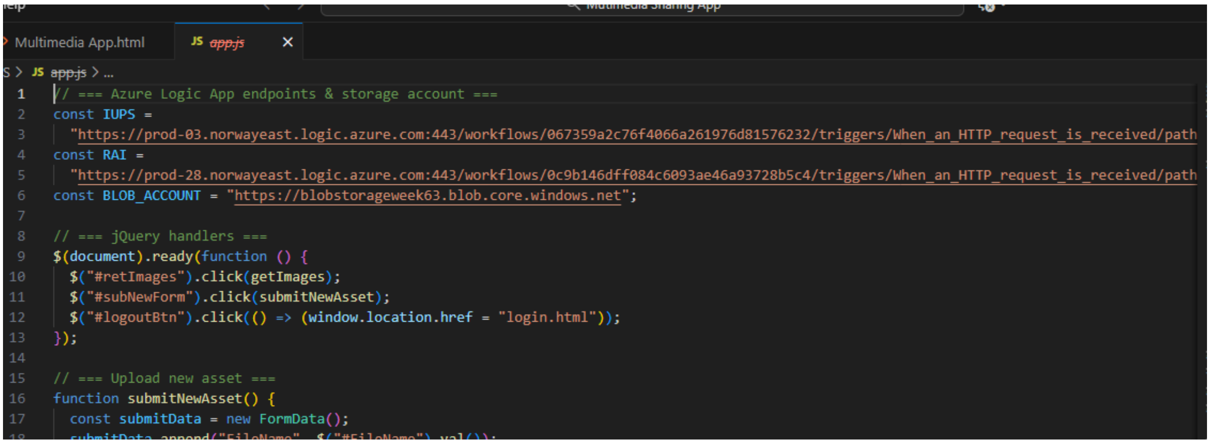
5. A form to add new photos.
6. A 'Submit' button with the ID "subNewForm" to take the contents of the form and send it to the IUPS API. This has an event handler specified in the app.js file.
7. A 'View Images" button with the ID "retImages" to view the images stored in the current database by accessing the RAI API and generating HTML on the fly. This has an event handler specified in the app.js file.
8. An Image display area with the ID "ImageList" this is used to display the HTML generated by the view images routine.

By default, this web app will not work. It needs additional code and variables specified in the app.js file, which has placeholders for the relevant items. Open this file in the code editor of your choice.

Modifying App.js

Providing REST URIs

Once you have the app.js file open, it is necessary to add some details about your REST endpoint. These are the URIs for IUPS and RAI. The necessary lines to be changed are 2,4 and 6. You should have a note of these URIs from the previous activities.



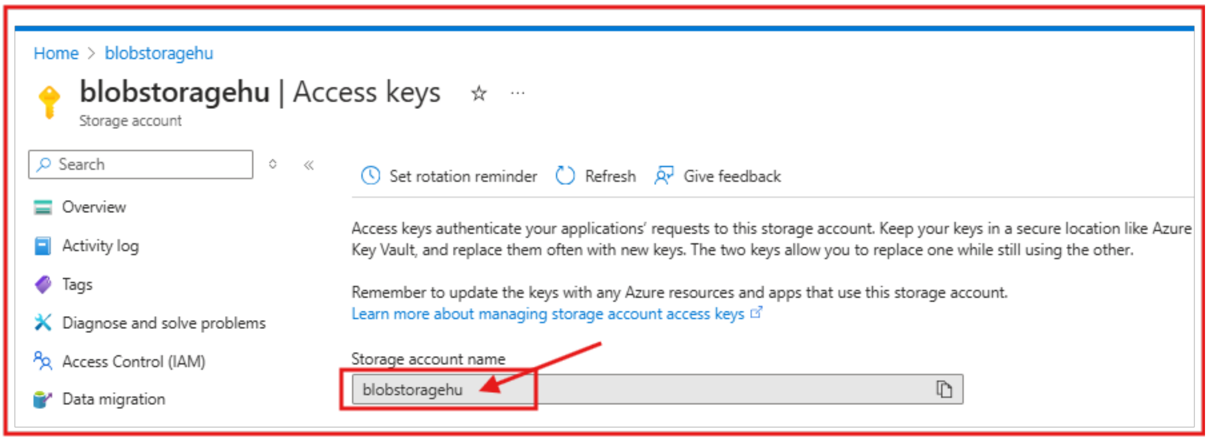
Providing the BASE URI for the storage account

Line 6: BLOB_ACCOUNT = After adding the REST API URIs, you must provide the Base URI for the blob storage you created in week6 practical.

This follows the form: https://**{X}**.blob.core.windows.net

This may be located in the overview panel for your blob storage under access keys. This is typically also the name of your blob storage account as shown in azure.

In our case, the blob storage name is: **blobstoragehu**.



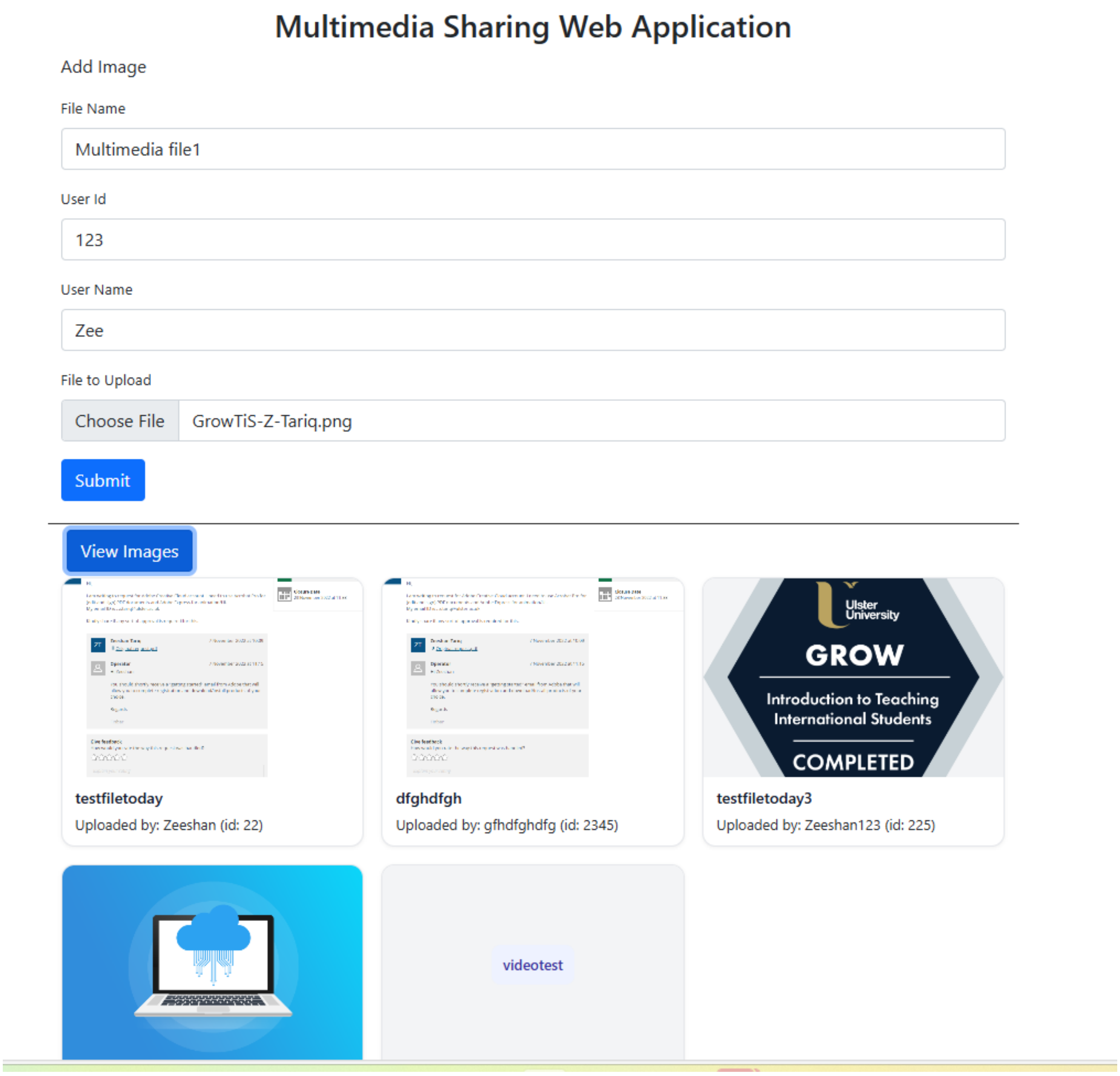
The {x} in the template string needs to be replaced with that, producing:

https:// blobstoragehu.blob.core.windows.net

Line 2: IUPS = This is URI of your IUPS logic app from Week-6 lab.

Line 4: RAI = This is URI of your RIA logic app from Week-6 lab.

Now save the app.js file. Open the html file and run the app.



Further thoughts

- These apps are not built to provide 100% functionalities; these are just supporting material for CW2. Minimal requirements of multimedia sharing are covered but that is not what you should aim for, for your CW2.
- Feel free to build your CW2 using these Web Apps as a baseline application.
- Please tailor these according to your use case and do not submit the same applications as part of your CW2.
- You can add as many functionalities/buttons/media types as you like for your CW2.